

PCEBENCH: A Multi-dimensional Benchmark for Evaluating Large Language Models in Parallel Code Generation

Le Chen*, Nesreen Ahmed[†], Mihai Capotă[‡], Ted Willke[§], Niranjan Hasabnis[¶], Ali Jannesari*

*Iowa State University, Ames, USA, {lechen, jannesar}@iastate.edu

[†]Cisco AI Research, San Jose, USA, nesahmed@cisco.com

[‡]Intel Labs, Hillsboro, USA, mihai.capota@intel.com

[§]DataStax, Santa Clara, USA, ted.willke@datastax.com

[¶]Code Metal, San Jose, USA, niranjan@codemetal.ai

Abstract—The increasing complexity of software systems and advancements in hardware architectures have intensified the demand for efficient parallel code generation. While parallel programming offers significant performance benefits, it requires extensive expertise and effort due to the intricacies of synchronization, data management, and optimizations. To address these challenges, recent studies have explored the application of machine learning (ML) techniques in parallel code generation, aiming to reduce manual efforts and enhance performance outcomes. Large language models (LLMs) have recently revolutionized natural language processing (NLP) and demonstrated remarkable capabilities in code generation. However, evaluating their ability to generate high-performance parallel code presents unique challenges. Unlike sequential code evaluation, the evaluation of LLM-generated parallel code requires consideration of not only correctness but also efficiency and scalability in utilizing parallel resources. Concretely, existing benchmarks for LLM-generated parallel code evaluation are limited in size and scope compared to their sequential counterparts. To address this evaluation gap, we introduce PCEBENCH, a novel benchmark designed to assess LLMs' capabilities in generating parallel code. PCEBENCH focuses on multi-tasking and multi-dimensional performance evaluation, leveraging an LLM-based approach to generate verified prompts for parallel code generation. The benchmark incorporates scripts compatible with compilers and data race checkers, enabling comprehensive testing across critical dimensions such as compilability, executability, code self-correctness, functional correctness, data race detection, and speedup over serial implementations. By examining these multiple dimensions, PCEBENCH not only facilitates a thorough evaluation of LLMs in parallel code generation but also provides valuable insights for developers to enhance model performance in this challenging task. This comprehensive approach contributes to advancing the field of automated parallel programming and supports the development of more efficient and scalable software systems.

Index Terms—large language model, parallel code generation, benchmark, evaluation, LLM agent

I. INTRODUCTION

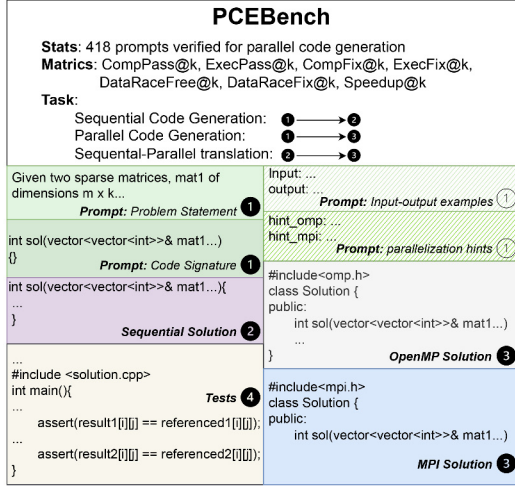
The rapid advancement of large language models (LLMs) has led to significant breakthroughs in various domains, including code-related tasks [1], [2]. Code LLMs such as StarCoder [3], CodeLLama [4], and DeepSeek-coder [5] have demonstrated remarkable capabilities across a wide range of

programming tasks, from code generation and completion to bug detection and API recommendation. These advancements have shown immense potential to enhance developer productivity and code quality. However, as LLMs continue to evolve, two critical challenges emerge: first, how to effectively evaluate the quality of LLM-generated code, especially in specialized and complex domains such as High-Performance Computing (HPC) [6]; and second, how to assess LLM performance across different stages of code generation, particularly in light of recent developments in agent-based frameworks that decompose complex problems into multiple reasoning steps.

HPC, with its reliance on parallel computing, presents a unique and crucial application area for LLMs. Parallel programming models like OpenMP for shared memory systems and MPI (Message Passing Interface) for distributed memory systems are fundamental to modern HPC, enabling the efficient execution of complex computational tasks across multiple processing units. The generation of correct, efficient, and scalable parallel code using these models is essential for leveraging the full potential of HPC systems. However, this task is inherently challenging due to the complexity of parallel algorithms, the need for precise synchronization, and the risk of concurrency issues such as race conditions.

Despite the importance of parallel programming in HPC, there has been limited research on evaluating the ability of LLMs to generate parallel code that meets the stringent demands of HPC applications [7]. Popular benchmarks, such as HumanEval [8] and MBPP [9], primarily focus on sequential code generation and fail to capture the unique challenges associated with parallel programming. Although the recent ParEval benchmark [10] represents a step towards filling this void in parallel code evaluation, it falls short in several key aspects. Compared to most machine learning benchmarks, ParEval is limited in the size of the dataset and lacks the granularity needed to provide insight into the various stages of parallel code generation.

To address these limitations, we propose PCEBENCH (Parallel Code Evaluation Bench), a novel benchmark specifically designed to evaluate LLM performance in parallel code



Parallel Code Generation using PCEBench

Prompt: Utilize prompts from PCEBench to instruct LLMs in generating parallel C/C++ code, with support for both MPI and OpenMP.

Strategy: Incorporate optional components to enhance flexibility in the code generation process.



Comprehensive Evaluation using PCEBench Scripts

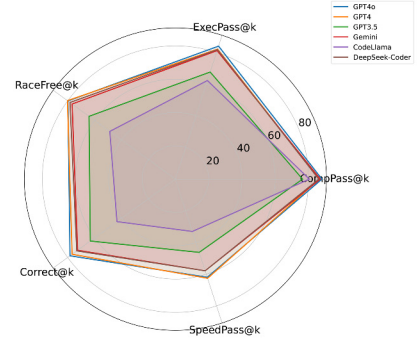


Fig. 1. Illustration of PCEBENCH. Left: PCEBENCH consists of structured prompts that have been verified for generating parallel code using OpenMP or MPI. The sketched boxes indicate optional components within the prompts. PCEBENCH also includes reference solutions and test programs. The detailed design of PCEBENCH is discussed in Section III. Middle: PCEBENCH includes scripts to run various parallel code generation strategies. Right: PCEBENCH performs comprehensive evaluations based on our designed metrics, automatically generating results.

generation and related tasks. PCEBENCH represents a significant advancement over existing approaches through several key contributions:

- 1) Designed specifically for parallel code generation: PCEBENCH comprises 621 prompts (409 for OpenMP and 212 for MPI) from various resources. Each of them is validated to be able to generate parallel C/C++ code and comes with validated sequential, OpenMP, and MPI solutions.
- 2) Multi-dimensional evaluation: PCEBENCH employs a comprehensive set of metrics, including but not limited to compilability (CompPass@k), executability (ExecPass@k), data race freedom (RaceFree@k), functional correctness (Correct@k), and performance improvement (SpeedPass@k). This multi-faceted approach provides a nuanced understanding of LLM capabilities in parallel programming.
- 3) Multi-task coverage: PCEBENCH evaluates LLMs on parallel code generation, sequential to parallel code translation, and repairing errors in parallel code. This multi-task approach offers insights into various aspects of LLM proficiency in parallel programming.
- 4) Extensible framework: PCEBENCH is designed to accommodate future developments in parallel programming and LLMs, ensuring its long-term relevance in this rapidly evolving field. With additional high-quality seed programs, we can extend PCEBENCH with increased numbers of OpenMP and MPI code, as well as code of other parallel languages.

The remainder of this paper is structured as follows: In Section III, we provide a detailed exposition of the design of PCEBENCH, outlining its key features. Section IV intro-

duces our innovative LLM agent-based approach for generating PCEBENCH data from diverse sources, highlighting the scalability and robustness of our benchmark. In Section V, we address three research questions through a comprehensive evaluation of state-of-the-art LLMs using PCEBENCH. Ultimately, our work aims to catalyze the broader adoption of LLMs in High-Performance Computing, thereby advancing the state-of-the-art in both artificial intelligence and high-performance parallel computing domains.

II. BACKGROUND

This section provides background information on large language models and their application in code generation. We further discuss the current benchmarks and metrics for evaluating LLM-generated HPC code.

A. Large Language Models in Code Generation

Built on the foundation of transformer architectures [11], large language models have been trained on massive amounts of text data, resulting in an extensive knowledge base and impressive language understanding and generation capabilities. These models, such as GPT-4 [12], have shown the ability to perform a wide range of NLP tasks. Recent advances have extended the application of large language models beyond natural language processing to code generation tasks. Researchers have developed systems that can generate, summarize, and even translate code across different programming languages by training these models on vast source code repositories. These code-focused LLMs, such as CodeLlama [4] and DeepSeek-Coder [5], have shown promising results in tasks such as automatically completing code snippets, generating entire functions or modules, and even translating code between different programming languages.

Recent work has expanded the application of LLMs to more complex problems through the use of LLM agent approaches. In this paradigm, multiple LLMs are orchestrated to work together, each playing a specific role in tackling different aspects of a complex task. These agent-based systems leverage the strengths of individual models, allowing them to communicate and collaborate in solving multi-faceted problems. For example, one model might focus on understanding the requirements of the problem, another on generating high-level solution strategies, and yet another on implementing the detailed code. This approach has shown promising results in addressing intricate software engineering tasks that require a combination of domain knowledge, problem-solving skills, and coding expertise. The advantage of this method lies in its ability to break down complex tasks into manageable sub-tasks, each handled by a specialized LLM agent. Consequently, there is a growing need for a comprehensive evaluation of LLMs in each phase of the code generation process. Such an evaluation would not only help in understanding the capabilities and limitations of different models but also in optimizing their roles within agent-based systems, ultimately leading to more effective and efficient code generation for complex problems.

B. LLM-generated Code Evaluation

In contrast to open text generation, where human preferences often take precedence — as highlighted by Ouyang et al. [13] — the code generation task focuses on the precision and fulfillment of user intentions, which are crucial for its practical implementation and use in production environments. As a result, evaluating code synthesis presents unique challenges in the era of LLM. Traditional token matching evaluation metrics such as BLEU [14] and ROUGE [15] show a weak correlation with human judgment [16] and overlook the functional correctness of the generated code [17], [18]. Recently, execution-based evaluation has gained increasing popularity, where code generated by models is tested through unit tests to verify its functional correctness. It leads to the development of several benchmarks, including HumanEval [8], MBPP [9], MBXP [19], CodeContests [20], and DS-1000 [21].

However, these benchmarks are designed for evaluating sequential code generation and do not focus on parallel code. Nichols et al. [10] addressed this gap by introducing the ParEval dataset, which evaluates the performance of generated parallel code, primarily in terms of speed. However, ParEval is limited in size compared to the previously mentioned benchmarks and focuses solely on the generated code, lacking insights into the parallel code generation process.

III. BENCHMARK DESIGN

The section describes the design criteria and guidelines for PCEBENCH and its evaluation metrics.

A. Design Criteria and Guidelines

PCEBENCH addresses the unique challenges associated with evaluating LLM-generated parallel code, focusing on

multiple dimensions of performance and correctness. We consider the following criteria and present our solutions:

Correctness: Our primary focus is ensuring the correctness of our data. We implement rigorous testing procedures with automated tools and manual inspections. This approach guarantees that all prompts in PCEBENCH can generate parallel codes with performance improvements. Verified parallel codes are listed as referenced parallel solutions in PCEBENCH.

Representativeness: To provide comprehensive insights, we source our data from both sequential algorithms and HPC benchmarks. Unlike previous approaches that focus solely on HPC data, our method enables a larger dataset by processing data from sequential code. This allows PCEBENCH to span different computing paradigms, from basic sequential operations to complex parallel processing.

Contamination-Free: Works such as [22] show that LLMs may have seen many programming problems in their training datasets, and the data contamination will affect the performance of LLMs. To ensure the integrity of our benchmark, we have rephrased problem statements and renamed functions and variables to use generic names. This approach minimizes the risk of LLMs simply recalling solutions they have seen before instead of genuinely solving the problems.

Multi-dimensional Evaluation: As the role of LLMs expands in specialized tasks within frameworks where they converse and collaborate, a comprehensive evaluation of LLMs in parallel code generation becomes crucial. We have designed a set of metrics to assess LLM capabilities across various dimensions, including 1) Generation of compilable and executable code, 2) Ensuring data race-free conditions in the generated code, 3) Maintaining functional correctness, and 4) Achieving performance improvements. Additionally, we evaluate the LLMs' ability to identify and correct issues in the generated code. This multi-faceted approach provides a holistic assessment of LLM performance in parallel programming tasks. We discuss these metrics in detail in Section III-D, offering insights into the strengths and limitations of LLMs in this complex domain.

Multi-task Capability: PCEBENCH incorporates a diverse range of prompt components, as detailed in Section III-B. This versatility enables our benchmark to perform a variety of tasks, spanning from parallel code generation to parallel code translation. Our multi-task approach ensures that PCEBENCH can evaluate LLM performance across a broad spectrum of parallel programming challenges, thus providing a comprehensive assessment of LLM capabilities in this domain. For a more detailed exploration of the specific tasks that can be performed with PCEBENCH, please refer to Section III-C.

Extensibility: The extensibility of PCEBENCH is twofold. First, our LLM agent-based approach has the advantage of automated data generation and verification with limited human labor required. It can adapt to new data formats and sources. Second, the framework can be integrated with other compilers or verification tools. This extensibility ensures that PCEBENCH remains relevant and useful as data and tool landscapes evolve. More details about data sources and tools

are discussed in Section IV-B and IV-C.

Interpretability: The benchmark provides clear metrics and explanations for each evaluation dimension, facilitating easy interpretation of results.

Ease of Use: PCEBENCH includes scripts to automate most tasks and provides a Makefile for the convenience of the HPC community. Results are automatically generated and can be presented in tables and figures.

Accessible: To enable maximal use of our benchmark suite, we will make our benchmark open-source and available on platforms such as GitHub and Huggingface.

B. PCEBENCH Example

The prompt data in PCEBENCH is structured to provide comprehensive information for parallel programming tasks. Listing 1 shows an example of a typical prompt, with explanations for each component:

- ID (*id*): This field provides a unique identifier for each problem in the PCEBENCH dataset. It allows for easy reference, tracking, and organization of problems within the benchmark suite.
- Problem Statement (*problem_statement*): This component provides a clear, detailed description of the programming task.
- Code Signature (*signature*): This specifies the expected interface for the solution, including input parameters and return type.
- Test Cases (*test_cases*): While omitted in this example for brevity, this section typically contains input-output pairs to validate the correctness of the generated solution.
- Sequential Solution (*sol_sequential*): This provides a reference sequential implementation of the solution.
- Parallel Solutions (*sol_omp* and *sol_mpi*): These fields contain reference parallel implementations using OpenMP and MPI, respectively.
- Parallelization Hints (*hint_omp* and *hint_mpi*): These hints provide high-level guidance on the parallelization strategies for each parallel programming model.
- Main function (*main_function*): The main function that invokes either the reference or the generated function.
- Source (*source*): This indicates the origin of the problem, which can be useful for categorization and analysis.

C. PCEBENCH Tasks

The primary objective of PCEBENCH is to assess the ability of large language models to generate parallel code. Specifically:

- **Parallel code generation:** Given a problem statement, a function signature, and optional hints, the LLM is tasked with producing efficient parallel implementations using OpenMP or MPI.

The generated code is evaluated on the basis of the metrics outlined in Section III-D, allowing for an assessment of the model's understanding of parallel programming concepts and its ability to apply them in solving computational problems.

```
{
  "id": 86,
  "problem_statement": "Write a C++ program to find a specific value in a two-dimensional grid of integers, where the grid has dimensions of m rows and n columns. The grid has the following characteristics: 1. Each row contains integers that are arranged in increasing order from the leftmost to the rightmost element. 2. Each column contains integers that are ordered in increasing sequence from the topmost to the bottommost element.",
  "signature" : "bool sol(vector<vector<int>>& matrix, int target)",
  "test_cases" : {
    "input": [omitted],
    "output": [omitted]
  },
  "sol_sequential": [omitted],
  "sol_omp": [omitted],
  "sol_mpi": [omitted],
  "hint_omp": "Parallel for loop over matrix rows. Use critical section to update shared result",
  "hint_mpi": "MPI_Scatter for data distribution. Local search on each process. MPI_Reduce to combine results",
  "main_function": [omitted],
  "source": "LeetCode"
}
```

Listing 1: Example of PCEBENCH data with structured, comprehensive components. Certain details, such as the solution code, are omitted for brevity.

While the focus of PCEBENCH is on parallel code evaluation, it also supports tasks such as:

- **Sequential Code Generation:** This serves as a baseline for comparing the LLM's performance in sequential versus parallel programming tasks and helps in assessing the model's general programming capabilities.
- **Sequential to Parallel Code Translation:** This task involves translating existing sequential code into parallel implementations. The LLM is provided with a sequential solution and is asked to create equivalent parallel versions using OpenMP or MPI. This evaluates the model's capability to identify parallelization opportunities and implement appropriate parallel constructs while preserving the original functionality.
- **Code Repair:** Our supplementary scripts enable tasks related to identifying and fixing issues in the generated code. This includes correcting syntax errors, logical errors, or inefficiencies in parallel implementations. The LLM can be prompted to review and improve its generated code or fix issues in the code provided to it.
- **Fixing Data Race Issues:** A critical aspect of parallel programming is avoiding data races. Our scripts facilitate tasks where an LLM is challenged to identify potential data races in the parallel code and propose fixes. This task evaluates the model's understanding of concurrent access patterns and synchronization mechanisms in par-

allel programming.

In this work, we focused on the main task of parallel code generation and conducted a systematic evaluation for the research questions discussed in Section V.

D. PCEBENCH Evaluation Metrics

To comprehensively assess the quality and performance of LLM-generated parallel code for OpenMP and MPI, we employ a set of metrics that cover various aspects of code correctness and efficiency. Each metric is evaluated using the $@k$ notation, where k represents the $top-k$ generations by the LLM. This approach allows us to assess the model's performance across multiple attempts, which is particularly relevant given the stochastic nature of LLM outputs. By considering the $top-k$ generations, we can better evaluate the LLM's consistency and capability in producing high-quality parallel code.

Compilability (CompPass@ k): This metric measures the rate at which the generated code is compiled successfully. CompPass@ k represents the percentage of code samples that compile without errors in the $top-k$ generations. It assesses an LLM's ability to produce syntactically correct code that adheres to OpenMP and MPI standards. The score of this metric is defined as:

$$\text{CompPass}@k = \frac{\text{Compiled code in } top-k}{\text{Total generated code}} \times 100$$

Executability (ExecPass@ k): ExecPass@ k evaluates the percentage of code samples that not only compile but also execute without runtime errors in the $top-k$ generations. This metric goes beyond syntactic correctness to ensure that the code can actually run in a parallel environment. The score of this metric is defined as:

$$\text{ExecPass}@k = \frac{\text{Executable code in } top-k}{\text{Total generated code}} \times 100$$

Functional Correctness (Correct@ k): Correct@ k measures the percentage of code samples in the $top-k$ generations that produce the correct output for given test cases. This metric assesses whether the parallelized code maintains the intended functionality of the original sequential algorithm. Correct@ k is defined as:

$$\text{Correct}@k = \frac{\text{Code without assertion errors in } top-k}{\text{Total generated code}} \times 100$$

Data Race Free (RaceFree@ k): RaceFree@ k evaluates the percentage of code samples in the $top-k$ generations that are free from data races. This is crucial for parallel programming, as data races can lead to non-deterministic behavior and incorrect results. We use static analysis tools to detect potential data races in the generated OpenMP and MPI code.

$$\text{RaceFree}@k = \frac{\text{Code without race issue in } top-k}{\text{Total generated code}} \times 100$$

Speedup (SpeedPass@ k): SpeedPass@ k measures the percentage of code samples in the $top-k$ generations that achieve

a speedup over the sequential version. This metric assesses the LLM's ability to not just parallelize code, but to do so in a way that improves performance. We define a threshold (e.g., 1.5x speedup) and count the samples that exceed this threshold.

$$\text{SpeedPass}@k = \frac{\text{parallel code with speedup in } top-k}{\text{Total generated code}} \times 100$$

These metrics provide a multi-dimensional evaluation of the LLM-generated parallel code, covering crucial aspects such as correctness, safety, and performance. By using these metrics, we can comprehensively assess the capabilities of LLMs in generating effective OpenMP and MPI code for parallel computing tasks.

IV. LLM AGENT-BASED DATA GENERATION

Figure 2 illustrates the pipeline employed to generate the PCEBENCH dataset. Our approach comprises three distinct phases: pre-processing, code generation, and verification. Each phase is orchestrated by a specialized LLM agent, ensuring a systematic and efficient data generation process. This section begins by introducing the LLM agent architecture utilized in this work, followed by a detailed description of each step in our pipeline.

A. LLM Agent

Recent studies, such as [23] and [24], have demonstrated the effectiveness of multi-agent systems in solving complex tasks through inter-agent communication. These LLM agent-based approaches, which integrate LLMs, human inputs, and specialized tools, show great promise for our task. Therefore, we constructed an LLM agent-based pipeline as shown in Figure 2 with a pre-define phase from the initial data processing through several validation stages, as outlined in later subsections.

Figure 3 shows the major components of the LLM agent used in our approach, including:

- **User request:** The initial input or query that defines the task or problem to be addressed by the LLM agent. This encompasses specific requirements for data generation, analysis parameters, and desired output formats.
- **Data input:** Information processed by the LLM agent, derived from two primary sources:
 - External data sources, including databases, APIs, and pre-existing datasets.
 - Feedback data from iterative loops within the pipeline, enabling result refinement and improvement based on previous outputs.
- **LLM:** The core AI component that processes user requests, interprets data, and generates appropriate responses or code. Functioning as the agent's brain, it leverages extensive knowledge to comprehend context and produce relevant output.
- **Code Executor:** A collection of scripts that compile and execute code determined and invoked by the agent. This component facilitates dynamic testing and validation of

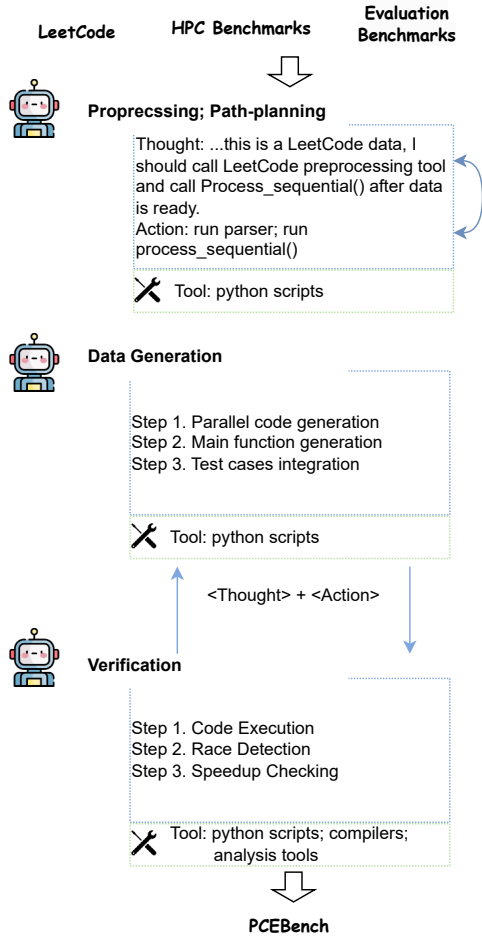


Fig. 2. The LLM agent-based pipeline for PCEBENCH data generation consists of three phases. The first agent pre-processes the data and selects pre-defined plans for different source data. The second agent interacts iteratively with the third agent to generate PCEBENCH data and verify its correctness.

generated code, ensuring functionality and correctness within the task context.

- Analysis tools: A suite of specialized tools and libraries designed to perform diverse data analysis tasks tailored to the specific requirements of the data generation process.
- Plan: Refers to the sequence or strategy the LLM agent generates or follows to accomplish a goal. It involves reasoning about the task, deciding what steps need to be taken, and forming a high-level blueprint before performing specific operations. For different input source data, we defined different plans to generate PCEBENCH data.
- Action: Refers to the concrete steps or operations that the agent takes to execute the plan. Actions involve interacting with the environment, invoking tools, or processing data based on the plan.

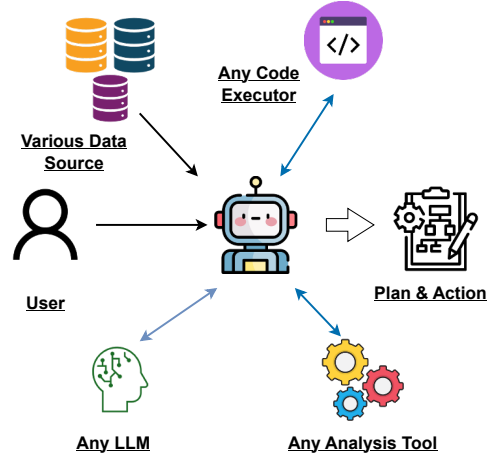


Fig. 3. The LLM agent in our approach monitors the environment and formulates plans to interact with other agents, invoke analysis tools, or execute scripts. The agents depicted in Figure 2 collaborate in an iterative loop to generate PCEBENCH data.

ReAct. Yao et al. [25] proposed ReAct, which integrates reasoning and acting capabilities into LLM applications. This approach allows LLMs not only to reason through tasks but also to interact with external environments or tools (such as APIs or databases) to gather additional information or execute actions based on their reasoning. In our work, the agent follows the ReAct design to generate PCEBENCH data.

B. Pre-Processing and Path Planning

1) *Dataset source:* Our data collection process focuses on diverse and high-quality sources to ensure comprehensive coverage of coding patterns and practices. Primary sources include:

- Sequential code repositories: We have collected programming problems from LeetCode¹, a popular online platform used to evaluate coding skills. Each LeetCode programming challenge contains well-defined task descriptions and input constraints, making it a great data source for building our benchmark.
- HPC benchmarks: To incorporate high-performance computing aspects, we sourced problems from two established HPC benchmarks, PolyBench [26] and OSU Micro-Benchmarks [27]. These benchmarks provide a range of computational kernels and verified parallelization solutions using OpenMP and MPI.
- Other evaluation benchmarks: Previous work such as ParEval [10] has prompts verified for parallel code generation.

2) *Data preprocessing:* In the initial phase, our LLM agent identifies and categorizes data from various sources and then implements targeted preprocessing strategies. This crucial step ensures data consistency and adherence to our contamination-

free guidelines. The preprocessing approach varies based on the data source:

- **LeetCode data:** The agent executes specialized scripts to parse LeetCode problems, systematically rephrasing them to align with our contamination-free guidelines. This process involves reformulating problem statements and modifying code signatures while preserving the core computational challenge.
 - **HPC benchmarks data:** For these complex computational tasks, the agent performs a two-step process:
 - 1) Generate a natural language problem statement that encapsulates the benchmark’s objectives and constraints.
 - 2) Clean the associated code by removing comments and refactoring variable names, ensuring the code remains functionally equivalent while minimizing potential contamination.
 - **Other evaluation benchmarks:** The agent rephrases task prompts and renames function signatures to comply with our contamination-free guidelines. This process maintains the essence of each benchmark while creating a standardized format across our dataset.
- 3) *Path Planning:* After preprocessing the data, our LLM agent executes plans based on the data source type. Each plan involves the data generation and verification agents in Figure 2.

C. Plans

The data generation and verification agents collaborate in an iterative loop for each plan. The data generation agent produces the necessary components for a given dataset and forwards them to the verification agent, which checks the data and provides feedback if errors are detected. The iteration limit is set to 10, after which manual review is required. The verification agent uses tools like GCC 14.2, Clang 18.1.8, and race detectors such as ThreadSanitizer and MUST.

The agents interact and retain memory, making the process largely automated with minimal manual intervention. Failed attempts are logged and excluded from future iterations. By tracking errors and historical outcomes, the agents continuously improve their ability to generate correct code more efficiently over time.

LeetCode. LeetCode problems have sequential solutions accompanied by well-explained problem statements. The objective of the plan is to generate parallel versions of these solutions using OpenMP and MPI. It also covers the main function with test cases included as assertions. The generated code is then sent to the verification agent, which ensures that the code is both functionally correct and capable of achieving speedup through parallelization.

HPC Benchmark. The objective for the HPC benchmark data is to modify the code by adding input-output assertions and generating its sequential version. For OpenMP code, this process is straightforward, simply requiring the removal of OpenMP pragmas. However, for MPI code, we leverage large language models (LLMs) to handle boundary cases, followed

by manual checks to ensure the correctness of the sequential version.

Other Evaluation Benchmarks. The objective for other evaluation benchmarks, such as ParEval, is to rename function and variable names to more generic terms and structure them in the format of PCEBENCH.

Once the agents successfully pass all verifications, the data is formatted according to the PCEBENCH data standard, as shown in Listing 1. Table I shows the diversity of PCEBENCH.

TABLE I
STATISTICS OF PCEBENCH DATA

	LeetCode	HPC Benchmark	Evaluation Benchmark	Total
OMP	318	31	60	409
MPI	110	42	60	212

V. EVALUATION

This section evaluates the effectiveness of LLMs using PCEBENCH. We address three research questions: assessing LLMs’ ability to generate parallel code (RQ1), their capability to fix errors (RQ2), and the impact of different prompt designs on the quality of generated code (RQ3).

A. Experiment Setup

All experiments were conducted on multiple nodes of an HPC cluster. Each node features a dual-socket AMD EPYC Rome 7402, with each socket housing 24 cores, along with 512 GB of DDR4-3200 RAM. Additionally, every node is equipped with four NVIDIA A100 40GB HBM2e Tensor Core GPUs, interconnected via NVLink3, optimizing performance for compute-intensive tasks.

We selected state-of-the-art general-purpose large language models such as GPT-3.5, GPT-4, GPT-4-turbo, and Gemini, alongside code-focused LLMs including CodeLlama and DeepSeek-Coder (33B). Inference was performed via API when available; otherwise, models were deployed locally using Hugging Face.

B. RQ1: Can LLMs Generate Correct and Efficient Parallel Code?

Research Question 1 (RQ1) focuses on evaluating the capability of large language Models to generate parallel code using OpenMP and MPI based on prompts provided in PCEBENCH. We also analyze the problems encountered in failed cases to gain insights into the challenges LLMs face in parallel programming tasks.

Methodology. We utilized the prompts from PCEBENCH to generate OpenMP and MPI code using large language models (LLMs). For each problem in the benchmark, we generated k samples of parallel code, with k values set to 1, 5, and 10. These samples were then evaluated using our defined metrics: CompPass@ k (compilation success), ExecPass@ k (execution success), RaceFree@ k (absence of data races), and Correct@ k (functional correctness).

Results and Observations. Given the evaluation results, we have below observations:

- Tables II and III present a comprehensive summary of the number and percentage of correct and efficient parallel OpenMP and MPI code generated by the evaluated LLMs. The results consistently demonstrate that GPT-4o outperforms other models, with GPT-4 following closely behind in performance.
- The data reveals a significant disparity in difficulty between MPI and OpenMP code generation, as evidenced by the marked difference in performance values. This underscores the increased complexity involved in generating MPI code compared to OpenMP code.
- An analysis of performance across different subsets of PCEBENCH indicates a substantial disparity in difficulty between HPC benchmark code generation and LeetCode code generation. The results clearly demonstrate that generating code for HPC benchmarks poses a significantly greater challenge for LLMs compared to LeetCode problems, highlighting the complexity and specialized knowledge required for high-performance computing tasks.
- Figures 4 and 5 employ bar charts to illustrate the impact of varying k values on the generation of correct and effective parallel code. The visualizations clearly demonstrate that the performance improvement is more pronounced when transitioning from $k=1$ to $k=5$, compared to the shift from $k=5$ to $k=10$. This observed trend in performance improvement suggests that there exists a threshold beyond which increasing the k value yields diminishing returns, particularly when utilizing simple prompts.
- Figures 6 and 7 utilize the evaluation script integrated within PCEBENCH to assess the performance of various models across the metrics established in this study. This comprehensive analysis decomposes the parallel code generation process into distinct stages, enabling a nuanced comparison of each model's capabilities across specific dimensions. The resulting insights prove invaluable for framework developers, facilitating the judicious selection of appropriate models for particular subtasks within the parallel code generation pipeline. By elucidating the strengths and weaknesses of each model in various aspects of parallel programming, these visualizations offer a sophisticated guide for optimizing the overall efficiency and effectiveness of automated parallel code generation systems.

C. RQ2: Can LLMs Generate Correct Code When Given Code with Errors?

Research Question 2 (RQ2) evaluates the capability of LLMs to identify and fix errors in previously generated parallel code for OpenMP and MPI. This question is crucial as it assesses the potential of LLMs not just in generating code but in debugging and improving parallel implementations.

Methodology We used the failed cases from RQ1 as our dataset for this experiment. The errors were categorized into

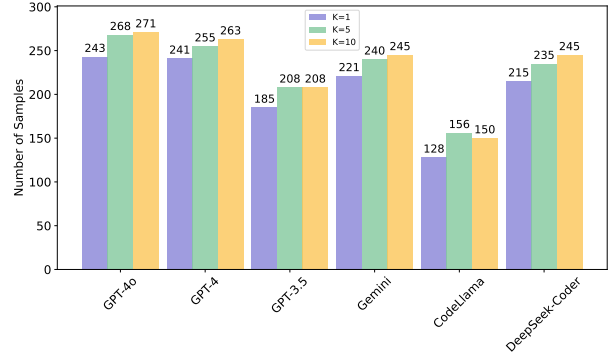


Fig. 4. Model performance comparison on OpenMP code generation for different k values.

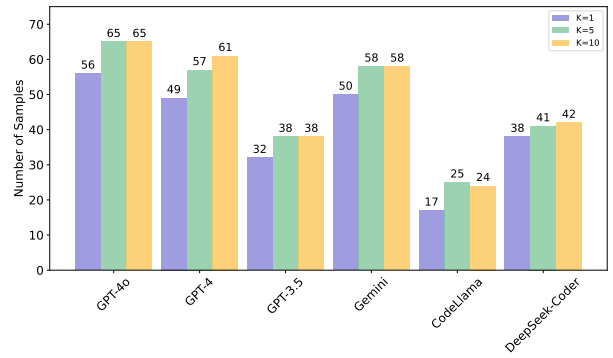


Fig. 5. Model performance comparison on MPI code generation for different k values.

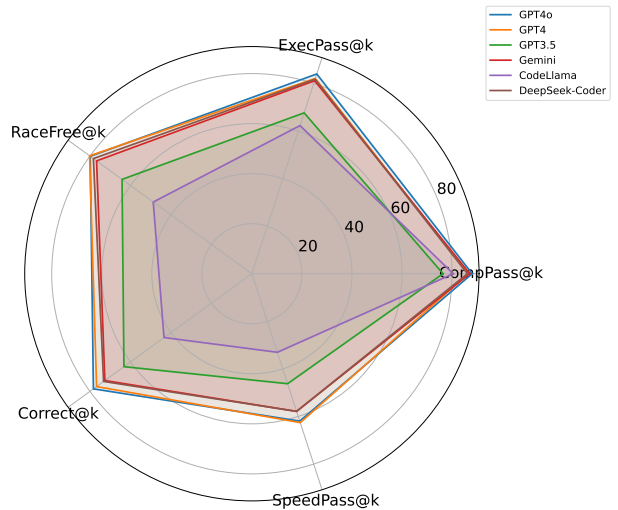


Fig. 6. Model performance on OpenMP code generation ($k=1$).

TABLE II
EVALUATION RESULTS OF LLMs IN GENERATING CORRECT AND EFFECTIVE OPENMP CODE.

Value of k	Model	LeetCode (318 samples)		HPC Benchmark (31 samples)		Evaluation Benchmark (60 samples)		Total (409 samples)	
		Number	Percentage	Number	Percentage	Number	Percentage	Number	Percentage
$k = 1$	GPT4o	197	61.9	12	38.7	34	56.7	243	59.4
	GPT4	199	62.6	11	35.5	31	51.7	241	58.9
	GPT3.5	147	46.2	7	22.6	31	51.7	185	45.2
	Gemini	184	57.9	7	22.6	30	50.0	221	54.0
	CodeLlama	105	33.0	5	16.1	17	28.3	128	31.3
	DeepSeek-Coder	184	57.9	9	29.0	22	36.7	215	52.6
$k = 5$	GPT4o	212	66.7	17	54.8	39	65.0	268	65.5
	GPT4	203	63.8	15	48.4	37	61.7	255	62.3
	GPT3.5	167	52.5	10	32.3	31	51.7	208	50.9
	Gemini	210	66.0	12	38.7	18	30.0	240	58.7
	CodeLlama	121	38.1	8	25.8	20	33.3	146	35.7
	DeepSeek-Coder	198	62.3	11	35.5	26	43.3	235	57.5
$k = 10$	GPT4o	214	67.3	17	54.8	40	66.7	271	66.3
	GPT4	211	66.4	15	48.4	37	61.7	263	64.3
	GPT3.5	167	52.5	10	32.3	31	51.7	208	50.9
	Gemini	213	67.0	13	41.9	19	31.7	245	59.9
	CodeLlama	122	38.4	8	25.8	20	33.3	150	36.7
	DeepSeek-Coder	207	65.1	12	38.7	26	43.3	245	59.9

TABLE III
EVALUATION RESULTS OF LLMs IN GENERATING CORRECT AND EFFECTIVE MPI CODE.

Value of k	Model	LeetCode (110 samples)		HPC Benchmark (42 samples)		Evaluation Benchmark (60 samples)		Total (212 samples)	
		number	percentage	number	percentage	number	percentage	number	percentage
$k = 1$	GPT4o	39	35.5	7	16.7	10	16.7	56	26.4
	GPT4	33	30.0	9	21.4	7	11.7	49	23.1
	GPT3.5	25	22.7	4	9.5	3	5.0	32	15.1
	Gemini	34	30.9	9	21.4	7	11.7	50	23.6
	CodeLlama	14	12.7	1	2.4	2	3.3	17	8.0
	DeepSeek-Coder	27	24.5	8	19.0	3	5.0	38	17.9
$k = 5$	GPT4o	44	13.8	10	23.8	11	18.3	65	30.7
	GPT4	39	12.3	9	21.4	9	15.0	57	26.9
	GPT3.5	30	9.4	5	11.9	3	5.0	38	17.9
	Gemini	42	13.2	9	21.4	7	11.7	58	27.4
	CodeLlama	22	6.9	1	2.4	2	3.3	25	11.8
	DeepSeek-Coder	30	9.4	8	19.0	3	5.0	41	19.3
$k = 10$	GPT4o	44	13.8	10	23.8	11	18.3	65	30.7
	GPT4	42	13.2	9	21.4	10	16.7	61	28.8
	GPT3.5	30	9.4	5	11.9	3	5.0	38	17.9
	Gemini	42	13.2	9	21.4	7	11.7	58	27.4
	CodeLlama	21	6.6	1	2.4	2	3.3	24	11.3
	DeepSeek-Coder	31	9.7	8	19.0	3	5.0	42	19.8

four types: Compilation errors, Runtime errors, Data race conditions, and Logical errors (incorrect output).

For each error type, we provided the LLM with The original problem statement, The erroneous code, and A description of the error (e.g., compiler output, runtime error message, or incorrect output).

Results and Observations. Given the evaluation results, we have below observations:

- Figures 8 and 9 illustrate the comparative performance of various models in addressing common errors in parallel programming. While GPT-4o consistently demonstrates superior performance across most categories, it is noteworthy that Gemini excels specifically in the task of rectifying OpenMP code compilation errors. This nuanced performance analysis provides valuable insights into the

specialized capabilities of different models within the domain of parallel code error correction.

- This experiment underscores the significance of PCEBENCH's structured design and the comprehensive metrics employed in this study. The findings extend beyond merely assessing the parallel code generation capabilities of LLMs. They provide crucial insights that enable LLM-based frameworks to judiciously select optimal models for their code-fixing agents. This granular analysis facilitates a more nuanced understanding of each model's strengths, potentially leading to the development of more efficient and effective automated code generation systems.

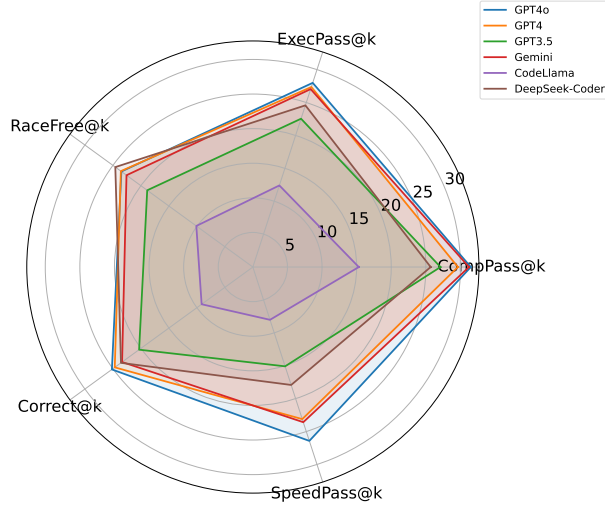


Fig. 7. Model performance on MPI code generation ($k=1$).

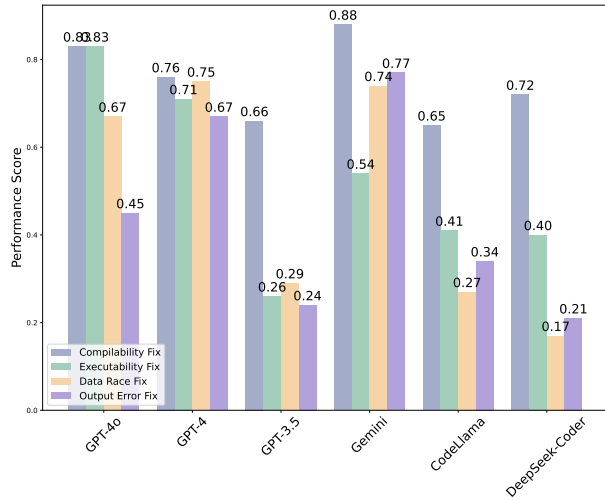


Fig. 8. Performance of LLMs in fixing common errors in OpenMP code generation ($k=1$).

D. RQ3: How Does Prompt Engineering Play a Role in Parallel Code Generation?

Research Question 3 (RQ3) investigates the impact of different prompt components and hint levels on the quality of parallel code generated by Large Language Models (LLMs). This question is crucial for understanding how to optimize prompts for better parallel code generation and to what extent additional information in prompts improves the output.

Methodology As shown in Figure 1, the structured design of PCEBENCH allows for optional components in the prompts. We systematically varied these components and the level of hints provided to create different experimental conditions. In this experiment, we utilized the following combinations of prompt components:

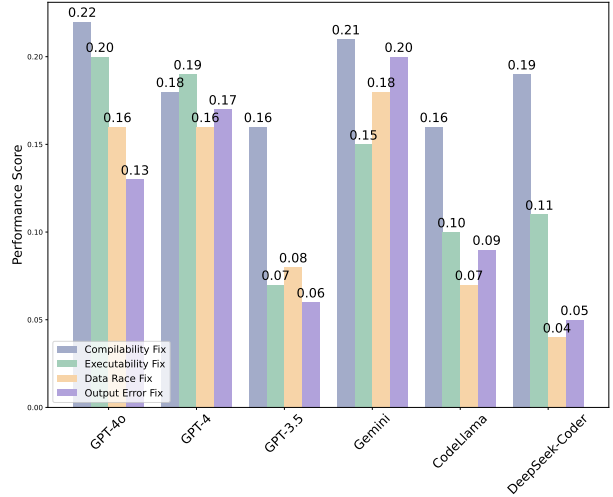


Fig. 9. Performance of LLMs in fixing common errors in MPI code generation ($k=1$).

- **P1:** Contains only the problem statement and code signature.
- **P2:** Contains P1 plus input-output examples.
- **P3:** Contains P1 plus parallelization hints.
- **P4:** Contains all components from the previous prompts.

We evaluated 100 randomly selected samples with 50 OpenMP and 50 MPI codes.

Results and Observations. Based on the evaluation results show in Figure 10, we have the following observations:

- Incorporating parallelization hints consistently improves LLM performance in parallel code generation.
- The improvement brought by input-output examples is significant for this task.
- Including all prompt components does not always result in better performance, suggesting the need for other prompt engineering techniques, such as chain-of-thought prompting.

VI. CONCLUSION

In this work, we introduced PCEBENCH, a multi-dimensional benchmark specifically designed to evaluate the capabilities of large language models in generating parallel code. With a focus on OpenMP and MPI, two fundamental paradigms in high-performance computing, PCEBENCH provides a robust framework for assessing key aspects of parallel code generation, including compilability, executability, correctness, data race freedom, and performance improvements.

Through a comprehensive set of experiments, we demonstrated that LLMs, while showing promise in generating parallel code, face significant challenges, particularly with MPI code generation. Our results highlighted the importance of multi-tasking and multi-dimensional evaluation, as well as the value of structured prompts and parallelization hints in improving LLM performance. Additionally, PCEBENCH's

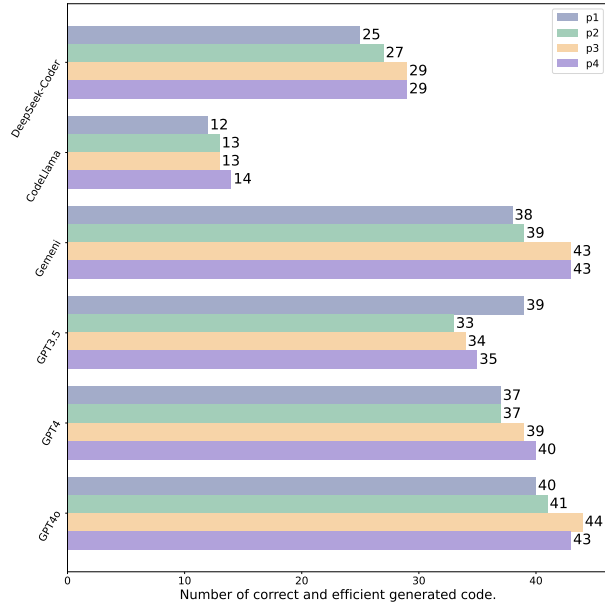


Fig. 10. Performance of LLMs in parallel code generation using prompt strategies p1 to p4. ($k=1$)

extensibility and automation framework allows it to evolve with future advancements in LLMs and parallel computing technologies.

Ultimately, PCEBENCH provides critical insights into the strengths and limitations of current LLMs, offering a valuable tool for researchers and developers aiming to optimize LLM-based systems for parallel code generation. We hope that PCEBENCH will serve as a foundational resource, guiding future research and innovation in this challenging domain.

VII. ACKNOWLEDGE

We want to thank NSF for their generous support in funding this project (#2211982). In addition, we extend our gratitude to Intel Labs for supporting this project.

REFERENCES

- [1] L. Chen, X. Ding, M. Emani, T. Vanderbruggen, P.-H. Lin, and C. Liao, "Data race detection using large language models," in *Proceedings of the SC'23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 215–223.
- [2] L. Chen, A. Bhattacharjee, N. Ahmed, N. Hasabnis, G. Oren, V. Vo, and A. Jannesari, "Ompgpt: A generative pre-trained transformer model for openmp," in *European Conference on Parallel Processing*. Springer, 2024, pp. 121–134.
- [3] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.
- [4] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [5] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: When the large language model meets programming—the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.

- [6] L. Chen, P.-H. Lin, T. Vanderbruggen, C. Liao, M. Emani, and B. De Supinski, "Lm4hpc: Towards effective language model application in high-performance computing," in *International Workshop on OpenMP*. Springer, 2023, pp. 18–33.
- [7] L. Chen, N. K. Ahmed, A. Dutta, A. Bhattacharjee, S. Yu, Q. I. Mahmud, W. Abebe, H. Phan, A. Sarkar, B. Butler *et al.*, "The landscape and challenges of hpc research and llms," *arXiv preprint arXiv:2402.02018*, 2024.
- [8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [9] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.
- [10] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, "Can large language models write parallel code?" in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, 2024, pp. 281–294.
- [11] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.
- [12] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [13] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.
- [14] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.
- [15] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Annual Meeting of the Association for Computational Linguistics*, 2004. [Online]. Available: <https://api.semanticscholar.org/CorpusID:964287>
- [16] M. Evtikhiev, E. Bogomolov, Y. Sokolov, and T. Bryksin, "Out of the bleu: how should we assess quality of the code generation models?" *Journal of Systems and Software*, vol. 203, p. 111741, 2023.
- [17] A. Eghbali and M. Pradel, "Crystalbleu: precisely and efficiently measuring the similarity of code," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.
- [18] N. Tran, H. Tran, S. Nguyen, H. Nguyen, and T. Nguyen, "Does bleu score work for code migration?" in *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. IEEE, 2019, pp. 165–176.
- [19] B. Athiwaratkun, S. K. Gouda, Z. Wang, X. Li, Y. Tian, M. Tan, W. U. Ahmad, S. Wang, Q. Sun, M. Shang *et al.*, "Multi-lingual evaluation of code generation models," *arXiv preprint arXiv:2210.14868*, 2022.
- [20] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [21] Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, W.-t. Yih, D. Fried, S. Wang, and T. Yu, "Ds-1000: A natural and reliable benchmark for data science code generation," in *International Conference on Machine Learning*. PMLR, 2023, pp. 18 319–18 345.
- [22] K. H. Naman Jain, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "Livecodebench: Holistic and contamination free evaluation of large language models for code," *arXiv preprint arXiv:2403.07974*, 2024.
- [23] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation framework," *arXiv preprint arXiv:2308.08155*, 2023.
- [24] I. E. Toubal, A. Avinash, N. G. Alldrin, J. Diab, W. Zhou, E. Luo, O. Stretcu, H. Xiong, C.-T. Lu, H. Zhou *et al.*, "Modeling collaborator: Enabling subjective vision classification with minimal human effort via llm tool-use," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 17 553–17 563.
- [25] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

- [26] J. Karimov, T. Rabl, and V. Markl, "Polybench: The first benchmark for polystores," in *Performance Evaluation and Benchmarking for the Era of Artificial Intelligence: 10th TPC Technology Conference, TPCTC 2018, Rio de Janeiro, Brazil, August 27–31, 2018, Revised Selected Papers 10*. Springer, 2019, pp. 24–41.
- [27] D. K. Panda, K. Tomko, K. Schulz, and A. Majumdar, "The mvapich project: Evolution and sustainability of an open source production quality mpi library for hpc," in *Workshop on Sustainable Software for Science: Practice and Experiences, held in conjunction with Int'l Conference on Supercomputing (WSSPE)*, 2013.